

#!/bin/bash

=====

=====

UNIT3D Auto-Installer v1.2 (Fixed)

**For UNIT3D Community Edition v9.2.0 (Laravel
12)**

Supports: Ubuntu 22.04 LTS / Ubuntu 24.04 LTS

=====

=====

set -e

Colors

RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

**Default values (customize these or pass as
arguments)**

```
DOMAIN=""
MYSQL_UNIT3D_PASS=""
OWNER_USERNAME="UNIT3D"
OWNER_EMAIL=""
OWNER_PASSWORD=""
TMDB_API_KEY=""
SMTP_HOST=""
SMTP_PORT="587"
SMTP_USER=""
SMTP_PASS=""
SMTP_FROM_ADDRESS=""
SMTP_FROM_NAME=""
INSTALL_PATH="/var/www/html"
PHP_VERSION=""
MEILI_KEY=""
```

FUNCTIONS

```
print_banner() {
echo -e "${BLUE}"
echo " "
echo "  UNIT3D Auto-Installer "
echo " Private Torrent Tracker "
echo " For Ubuntu 22.04 / 24.04 LTS "
echo " v1.2 (Fixed for 9.2.0) "
echo " "
echo -e "${NC}"
}

log_info() {
echo -e "${GREEN}[INFO]${NC} $1"
}

log_warn() {
echo -e "${YELLOW}[WARN]${NC} $1"
}

log_error() {
echo -e "${RED}[ERROR]${NC} $1"
}
```

```
check_root() {  
if [[ $EUID -ne 0 ]]; then  
log_error "This script must be run as root"  
exit 1  
fi  
}
```

```
check_os() {  
if [[ -f /etc/os-release ]]; then  
. /etc/os-release  
OS=$ID  
VERSION=$VERSION_ID  
else  
log_error "Cannot detect OS version"  
exit 1  
fi
```

```
if [[ "$OS" != "ubuntu" ]]; then  
log_error "This script only supports Ubuntu"  
exit 1  
fi  
  
case $VERSION in  
"22.04")  
PHP_VERSION="8.4"  
log_info "Detected Ubuntu 22.04 - Will install PHP $PHP_VERSION"  
;;  
"24.04")  
PHP_VERSION="8.4"  
log_info "Detected Ubuntu 24.04 - Will install PHP $PHP_VERSION"  
;;  
*)  
log_error "Ubuntu $VERSION is not supported. Use 22.04 or 24.04"  
exit 1  
;;  
esac
```

```
}  
  
generate_password() {  
openssl rand -base64 16 | tr -dc 'a-zA-Z0-9' | head -c 20  
}
```

```
interactive_setup() {  
echo ""  
log_info "Starting interactive setup..."  
echo ""
```

```
# Domain  
while [[ -z "$DOMAIN" ]]; do  
read -p "Enter your domain (e.g., tracker.example.com): " DOMAIN  
done  
  
# MySQL UNIT3D Password  
if [[ -z "$MYSQL_UNIT3D_PASS" ]]; then  
MYSQL_UNIT3D_PASS=$(generate_password)  
log_info "Generated MySQL UNIT3D password"  
fi  
  
# Owner details  
read -p "Owner username [OWNER_USERNAME]: " input  
OWNER_USERNAME="${input:-$OWNER_USERNAME}"  
  
while [[ -z "$OWNER_EMAIL" ]]; do  
read -p "Owner email: " OWNER_EMAIL
```

```

done

if [[ -z "$OWNER_PASSWORD" ]]; then
    OWNER_PASSWORD=$(generate_password)
    log_info "Generated owner password"
fi

# TMDB API Key (optional)
read -p "TMDB API Key (optional, press enter to skip): " TMDB_API_KEY

# SMTP Settings
echo ""
log_info "SMTP Configuration (for email notifications)"
read -p "SMTP Host (e.g., smtp.mailgun.org): " SMTP_HOST
read -p "SMTP Port [$SMTP_PORT]: " input
SMTP_PORT="${input:-$SMTP_PORT}"
read -p "SMTP Username: " SMTP_USER
read -s -p "SMTP Password: " SMTP_PASS
echo ""
read -p "From Email Address: " SMTP_FROM_ADDRESS
read -p "From Name [$DOMAIN]: " input
SMTP_FROM_NAME="${input:-$DOMAIN}"

echo ""
log_info "Configuration Summary:"
echo "  Domain: $DOMAIN"
echo "  Install Path: $INSTALL_PATH"
echo "  Owner: $OWNER_USERNAME ($OWNER_EMAIL)"
echo "  PHP Version: $PHP_VERSION"
echo ""
read -p "Continue with installation? [y/N]: " confirm
if [[ "$confirm" != "y" && "$confirm" != "Y" ]]; then
    log_info "Installation cancelled"
    exit 0
fi

```

```

}

```

```

install_dependencies() {
    log_info "Updating system packages..."
    apt update && apt upgrade -y

```

```

    log_info "Installing base dependencies..."
    apt install -y      software-properties-common      apt-transport-https      ca-certificates      curl
    wget      gnupg      lsb-release      unzip      git      acl      cron      net-tools

    # Add PHP repository
    log_info "Adding PHP repository..."
    add-apt-repository -y ppa:ondrej/php
    apt update

```

```

# Install PHP and extensions
log_info "Installing PHP $PHP_VERSION and extensions..."
apt install -y      php${PHP_VERSION}-fpm      php${PHP_VERSION}-cli      php${PHP_VERSION}-mysql
php${PHP_VERSION}-pgsql      php${PHP_VERSION}-sqlite3      php${PHP_VERSION}-redis      php${PHP_VERSION}-
memcached      php${PHP_VERSION}-curl      php${PHP_VERSION}-gd      php${PHP_VERSION}-imagick
php${PHP_VERSION}-mbstring      php${PHP_VERSION}-xml      php${PHP_VERSION}-zip      php${PHP_VERSION}-
bcmath      php${PHP_VERSION}-intl      php${PHP_VERSION}-soap      php${PHP_VERSION}-opcache
php${PHP_VERSION}-readline      php${PHP_VERSION}-common      php${PHP_VERSION}-igbinary
php${PHP_VERSION}-msgpack

# Install Nginx
log_info "Installing Nginx..."
apt install -y nginx

# Install MySQL
log_info "Installing MySQL..."
apt install -y mysql-server

# Install Redis
log_info "Installing Redis..."
apt install -y redis-server

# Install Supervisor
log_info "Installing Supervisor..."
apt install -y supervisor

# Install Node.js 20 (needed for laravel-echo-server)
log_info "Installing Node.js 20..."
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Install Bun (used for asset building)
log_info "Installing Bun..."
curl -fsSL https://bun.sh/install | bash
# Source bun into current shell
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"

# Verify bun is accessible
if ! command -v bun &> /dev/null; then
    log_error "Bun installation failed. Trying alternative path..."
    export PATH="/root/.bun/bin:$PATH"
fi

# Install Composer
log_info "Installing Composer..."
curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/bin --filename=composer

# Install Certbot for SSL
log_info "Installing Certbot..."
apt install -y certbot python3-certbot-nginx

# Install Meilisearch
log_info "Installing Meilisearch..."
curl -L https://install.meilisearch.com | sh
mv ./meilisearch /usr/local/bin/
chmod +x /usr/local/bin/meilisearch

# Install Laravel Echo Server globally
log_info "Installing Laravel Echo Server..."
npm install -g laravel-echo-server

```

```

}

```

```

configure_mysql() {
log_info "Configuring MySQL..."

```

```
# Start MySQL
systemctl start mysql
systemctl enable mysql

# On fresh Ubuntu install, root uses auth_socket (no password needed when run as root)
# Create UNIT3D database and user directly
mysql -e "CREATE DATABASE IF NOT EXISTS unit3d CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -e "DROP USER IF EXISTS 'unit3d'@'localhost';"
mysql -e "CREATE USER 'unit3d'@'localhost' IDENTIFIED WITH mysql_native_password BY '${MYSQL_UNIT3D_PASS}';"
mysql -e "GRANT ALL PRIVILEGES ON unit3d.* TO 'unit3d'@'localhost';"
mysql -e "FLUSH PRIVILEGES;"

log_info "MySQL configured successfully"
```

```
}
```

```
configure_redis() {
log_info "Configuring Redis..."
```

```
# Enable Redis
systemctl start redis-server
systemctl enable redis-server

# Configure Redis memory limits
sed -i 's/^# maxmemory .*/maxmemory 256mb/' /etc/redis/redis.conf
grep -q "^maxmemory " /etc/redis/redis.conf || echo "maxmemory 256mb" >> /etc/redis/redis.conf
grep -q "^maxmemory-policy" /etc/redis/redis.conf || echo "maxmemory-policy allkeys-lru" >> /etc/redis/redis.conf

systemctl restart redis-server
log_info "Redis configured successfully"
```

```
}
```

```
configure_meilisearch() {
log_info "Configuring Meilisearch..."
```

```
# Generate API key
MEILI_KEY=$(openssl rand -hex 16)

# Create data directory
mkdir -p /var/lib/meilisearch

# Create systemd service
cat > /etc/systemd/system/meilisearch.service << EOF
```

```
[Unit]
```

```
Description=Meilisearch
```

```
After=network.target
```

```
[Service]
```

```
User=root
```

```
ExecStart=/usr/local/bin/meilisearch --http-addr 127.0.0.1:7700 --master-key ${MEILI_KEY} --db-path /var/lib/meilisearch --env
production
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
systemctl daemon-reload
systemctl enable meilisearch
systemctl start meilisearch

log_info "Meilisearch configured successfully"
```

```
}

configure_php() {
log_info "Configuring PHP $PHP_VERSION..."
```

```
PHP_INI="/etc/php/${PHP_VERSION}/fpm/php.ini"
PHP_FPM_CONF="/etc/php/${PHP_VERSION}/fpm/pool.d/www.conf"

# PHP.ini optimizations
sed -i "s/upload_max_filesize = .*/upload_max_filesize = 256M/" $PHP_INI
sed -i "s/post_max_size = .*/post_max_size = 256M/" $PHP_INI
sed -i "s/memory_limit = .*/memory_limit = 512M/" $PHP_INI
sed -i "s/max_execution_time = .*/max_execution_time = 600/" $PHP_INI
sed -i "s/max_input_time = .*/max_input_time = 600/" $PHP_INI
sed -i "s/;date.timezone = .*/date.timezone = UTC/" $PHP_INI
sed -i "s/;cgi.fix_pathinfo=1/cgi.fix_pathinfo=0/" $PHP_INI

# OPcache settings
cat >> $PHP_INI << 'EOF'
```

```
; OPcache settings for UNIT3D
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=10000
opcache.revalidate_freq=0
opcache.validate_timestamps=0
opcache.save_comments=1
opcache.fast_shutdown=1
EOF
```

```
# PHP-FPM pool configuration
sed -i "s/;request_terminate_timeout = .*/request_terminate_timeout = 600/" $PHP_FPM_CONF
sed -i "s/pm.max_children = .*/pm.max_children = 50/" $PHP_FPM_CONF
sed -i "s/pm.start_servers = .*/pm.start_servers = 10/" $PHP_FPM_CONF
sed -i "s/pm.min_spare_servers = .*/pm.min_spare_servers = 5/" $PHP_FPM_CONF
sed -i "s/pm.max_spare_servers = .*/pm.max_spare_servers = 20/" $PHP_FPM_CONF

systemctl restart php${PHP_VERSION}-fpm
log_info "PHP configured successfully"
```

```
}

configure_nginx() {
log_info "Configuring Nginx..."
```

```
# Create Nginx config for UNIT3D with Socket.IO support
cat > /etc/nginx/sites-available/unit3d << EOF
```

```
server {
listen 80;
listen [::]:80;
server_name ${DOMAIN} www.${DOMAIN};
```

```
root ${INSTALL_PATH}/public;
index index.php;

client_max_body_size 256M;

# Security headers
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

```

# Gzip compression
gzip on;
gzip_comp_level 5;
gzip_min_length 256;
gzip_proxied any;
gzip_vary on;
gzip_types
    application/javascript
    application/json
    application/xml
    application/rss+xml
    image/svg+xml
    text/css
    text/javascript
    text/plain
    text/xml;

location / {
    try_files $uri $uri/ /index.php?$query_string;
}

# Socket.IO for Laravel Echo Server (Chat)
location /socket.io {
    proxy_pass http://127.0.0.1:6001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 60s;
    proxy_send_timeout 60s;
}

location ~ /\.php$ {
    fastcgi_split_path_info ^(.+\.php)(/.+)\$;
    fastcgi_pass unix:/var/run/php/php${PHP_VERSION}-fpm.sock;
    fastcgi_index index.php;
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    fastcgi_param PATH_INFO $fastcgi_path_info;
    fastcgi_read_timeout 600;
    fastcgi_buffers 16 16k;
    fastcgi_buffer_size 32k;
}

location ~ /\.(!well-known).* {
    deny all;
}

location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)\$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    access_log off;
}

# Deny access to sensitive files
location ~* (\.env|\.git|composer\.json|composer\.lock|package\.json|package-lock\.json|\.htaccess)\$ {
    deny all;
}

```

}
 EOF

```

# Enable the site
sudo systemctl restart nginx

```



```
rm -f /etc/nginx/sites-enabled/default
ln -sf /etc/nginx/sites-available/unit3d /etc/nginx/sites-enabled/
```

```
# Test and reload Nginx
```

```
nginx -t
systemctl restart nginx
systemctl enable nginx
```

```
log_info "Nginx configured successfully"
```

```
}
```

```
install_unit3d(){
log_info "Installing UNIT3D..."
```

```
# Add safe directory for git
```

```
git config --global --add safe.directory ${INSTALL_PATH}
```

```
# Remove existing installation
```

```
rm -rf ${INSTALL_PATH}
```

```
# Clone UNIT3D Community Edition
```

```
git clone https://github.com/HDInnovations/UNIT3D-Community-Edition.git ${INSTALL_PATH}
cd ${INSTALL_PATH}
```

```
# Checkout the specific v9.2.0 release
```

```
log_info "Checking out v9.2.0..."
```

```
git checkout v9.2.0 2>/dev/null || git checkout main
```

```
log_info "Checked out UNIT3D version: $(git describe --tags 2>/dev/null || git rev-parse --short HEAD)"
```

```
# Create .env file from example
```

```
cp .env.example .env
```

```
# Configure .env - using sed to replace existing values
```

```
sed -i "s|APP_NAME=.*|APP_NAME=\"${DOMAIN}\"|" .env
```

```
sed -i "s|APP_ENV=.*|APP_ENV=prod|" .env
```

```
sed -i "s|APP_DEBUG=.*|APP_DEBUG=false|" .env
```

```
sed -i "s|APP_URL=.*|APP_URL=https://${DOMAIN}|" .env
```

```
# Database
```

```
sed -i "s|DB_CONNECTION=.*|DB_CONNECTION=mysql|" .env
```

```
sed -i "s|DB_HOST=.*|DB_HOST=127.0.0.1|" .env
```

```
sed -i "s|DB_PORT=.*|DB_PORT=3306|" .env
```

```
sed -i "s|DB_DATABASE=.*|DB_DATABASE=unit3d|" .env
```

```
sed -i "s|DB_USERNAME=.*|DB_USERNAME=unit3d|" .env
```

```
sed -i "s|DB_PASSWORD=.*|DB_PASSWORD=${MYSQL_UNIT3D_PASS}|" .env
```

```
# Echo address for chat (Vite env var)
```

```
sed -i "s|VITE_ECHO_ADDRESS=.*|VITE_ECHO_ADDRESS=https://${DOMAIN}|" .env
```

```
# Broadcasting, Cache, Session, Queue (matching .env.example variable names)
```

```
sed -i "s|BROADCAST_CONNECTION=.*|BROADCAST_CONNECTION=redis|" .env
```

```
sed -i "s|CACHE_STORE=.*|CACHE_STORE=redis|" .env
```

```
sed -i "s|SESSION_DRIVER=.*|SESSION_DRIVER=redis|" .env
```

```
sed -i "s|SESSION_CONNECTION=.*|SESSION_CONNECTION=session|" .env
```

```
sed -i "s|SESSION_SECURE_COOKIE=.*|SESSION_SECURE_COOKIE=true|" .env
```

```
sed -i "s|QUEUE_CONNECTION=.*|QUEUE_CONNECTION=redis|" .env
```

```
# Redis
```

```
sed -i "s|REDIS_HOST=.*|REDIS_HOST=127.0.0.1|" .env
```

```
sed -i "s|REDIS_PASSWORD=.*|REDIS_PASSWORD=null|" .env
```

```
sed -i "s|REDIS_PORT=.*|REDIS_PORT=6379|" .env
```

```
# Meilisearch
```

```
sed -i "s|MEILISEARCH_HOST=.*|MEILISEARCH_HOST=http://127.0.0.1:7700|" .env
```

```
sed -i "s|MEILISEARCH_KEY=.*|MEILISEARCH_KEY=${MEILI_KEY}|" .env
```

```

# Mail
sed -i "s|MAIL_MAILER=. *|MAIL_MAILER=smtp|" .env
sed -i "s|MAIL_HOST=. *|MAIL_HOST=${SMTP_HOST}|" .env
sed -i "s|MAIL_PORT=. *|MAIL_PORT=${SMTP_PORT}|" .env
sed -i "s|MAIL_USERNAME=. *|MAIL_USERNAME=${SMTP_USER}|" .env
sed -i "s|MAIL_PASSWORD=. *|MAIL_PASSWORD=${SMTP_PASS}|" .env
sed -i "s|MAIL_ENCRYPTION=. *|MAIL_ENCRYPTION=tls|" .env
sed -i "s|MAIL_FROM_ADDRESS=. *|MAIL_FROM_ADDRESS=${SMTP_FROM_ADDRESS}|" .env
sed -i "s|MAIL_FROM_NAME=. *|MAIL_FROM_NAME=${SMTP_FROM_NAME}|" .env

# TMDB
if [[ -n "$TMDB_API_KEY" ]]; then
    sed -i "s|TMDB_API_KEY=. *|TMDB_API_KEY=${TMDB_API_KEY}|" .env
fi

# Owner credentials
sed -i "s|DEFAULT_OWNER_NAME=. *|DEFAULT_OWNER_NAME=${OWNER_USERNAME}|" .env
sed -i "s|DEFAULT_OWNER_EMAIL=. *|DEFAULT_OWNER_EMAIL=${OWNER_EMAIL}|" .env
sed -i "s|DEFAULT_OWNER_PASSWORD=. *|DEFAULT_OWNER_PASSWORD=${OWNER_PASSWORD}|" .env

# Set ownership BEFORE composer install so www-data can write
chown -R www-data:www-data ${INSTALL_PATH}

# Install Composer dependencies (as www-data)
log_info "Installing Composer dependencies..."
sudo -u www-data composer install --no-dev --optimize-autoloader --no-interaction --working-dir=${INSTALL_PATH}

# Install NPM dependencies and build assets using Bun (as www-data)
log_info "Installing NPM dependencies and building assets with Bun..."
export BUN_INSTALL="$HOME/.bun"
export PATH="$BUN_INSTALL/bin:$PATH"
cd ${INSTALL_PATH}
sudo -E -u www-data bash -c "export PATH=$BUN_INSTALL/bin:\$PATH && cd ${INSTALL_PATH} && bun install && bun run build"

# If bun as www-data fails, fallback to running as root then fixing permissions
if [[ $? -ne 0 ]]; then
    log_warn "Bun as www-data failed, running as root and fixing permissions..."
    cd ${INSTALL_PATH}
    bun install
    bun run build
    chown -R www-data:www-data ${INSTALL_PATH}
fi

# Set permissions
chmod -R 755 ${INSTALL_PATH}
chmod -R 775 ${INSTALL_PATH}/storage
chmod -R 775 ${INSTALL_PATH}/bootstrap/cache

# Generate app key using artisan (the only correct way)
log_info "Generating application key..."
sudo -u www-data php artisan key:generate --force

# Run migrations and seed
log_info "Running database migrations and seeding..."
sudo -u www-data php artisan migrate --force --seed

# Create storage link
sudo -u www-data php artisan storage:link

# Import Meilisearch indexes
log_info "Setting up Meilisearch indexes..."
sudo -u www-data php artisan scout:sync-index-settings || true
sudo -u www-data php artisan scout:import "App\Models\Torrent" || true

# Cache configuration for production
log_info "Optimizing application..."

```

```
log_info "Optimizing application..."
sudo -u www-data php artisan config:cache
sudo -u www-data php artisan route:cache
sudo -u www-data php artisan view:cache
```

```
log_info "UNIT3D installed successfully"
```

```
}

configure_laravel_echo_server() {
log_info "Configuring Laravel Echo Server..."
```

```
# Create config file
cat > ${INSTALL_PATH}/laravel-echo-server.json << EOF
```

```
{
"authHost": "https://${DOMAIN}",
"authEndpoint": "/broadcasting/auth",
"clients": [],
"database": "redis",
"databaseConfig": {
"redis": {
"host": "127.0.0.1",
"port": "6379"
}
},
"devMode": false,
"host": "127.0.0.1",
"port": "6001",
"protocol": "http",
"socketio": {},
"sslCertPath": "",
"sslKeyPath": "",
"sslCertChainPath": "",
"sslPassphrase": "",
"subscribers": {
"http": true,
"redis": true
},
"apiOriginAllow": {
"allowCors": true,
"allowOrigin": "https://${DOMAIN}",
"allowMethods": "GET, POST",
"allowHeaders": "Origin, Content-Type, X-Auth-Token, X-Requested-With, Accept, Authorization, X-CSRF-TOKEN, X-Socket-Id"
}
}
}
EOF
```

```
chown www-data:www-data ${INSTALL_PATH}/laravel-echo-server.json
```

```
log_info "Laravel Echo Server configured successfully"
```

```
}

configure_supervisor() {
log_info "Configuring Supervisor for queue workers and chat..."
```

```
cat > /etc/supervisor/conf.d/unit3d.conf << EOF
```

```
[program:unit3d-queue]
process_name=%(program_name)s_%(process_num)02d
command=php ${INSTALL_PATH}/artisan queue:work redis --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=4
redirect_stderr=true
stdout_logfile=${INSTALL_PATH}/storage/logs/queue.log
stopwaitsecs=3600
```

```
[program:unit3d-echo]
process_name=%(program_name)s_%(process_num)02d
command=laravel-echo-server start --dir=${INSTALL_PATH}
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=1
redirect_stderr=true
stdout_logfile=${INSTALL_PATH}/storage/logs/echo.log
EOF
```

```
supervisorctl reread
supervisorctl update
supervisorctl start all

log_info "Supervisor configured successfully"
```

```
}

configure_cron() {
log_info "Configuring Laravel scheduler cron..."
```

```
# Add Laravel scheduler to crontab
(crontab -l 2>/dev/null | grep -v "artisan schedule:run"; echo "* * * * * cd ${INSTALL_PATH} && php artisan
schedule:run >> /dev/null 2>&1") | crontab -

log_info "Cron configured successfully"
```

```
}

setup_ssl() {
log_info "Setting up SSL with Let's Encrypt..."
```

```
certbot --nginx -d ${DOMAIN} -d www.${DOMAIN} --non-interactive --agree-tos -m ${OWNER_EMAIL} --redirect

# After SSL is set up, clear config cache and re-cache (URLs change to https)
cd ${INSTALL_PATH}
sudo -u www-data php artisan config:cache

log_info "SSL configured successfully"
```

```
}

save_credentials() {
CREDS_FILE="/root/unit3d-credentials.txt"
```

```
cat > $CREDS_FILE << EOF
```

=====

UNIT3D Installation Credentials

Generated: \$(date)

=====

DOMAIN: \${DOMAIN}

URL: https://\${DOMAIN}

OWNER LOGIN:

Username: \${OWNER_USERNAME}

Email: \${OWNER_EMAIL}

Password: \${OWNER_PASSWORD}

MYSQL:

Root: Uses socket auth (no password, run mysql as root)

UNIT3D Database: unit3d

UNIT3D User: unit3d

UNIT3D Password: \${MYSQL_UNIT3D_PASS}

MEILISEARCH:

API Key: \${MEILI_KEY}

INSTALLATION PATH: \${INSTALL_PATH}

PHP VERSION: \${PHP_VERSION}

IMPORTANT FILES:

.env: \${INSTALL_PATH}/.env

Nginx: /etc/nginx/sites-available/unit3d

Supervisor: /etc/supervisor/conf.d/unit3d.conf

Echo Server: \${INSTALL_PATH}/laravel-echo-server.json

USEFUL COMMANDS:

Restart PHP-FPM: systemctl restart php\${PHP_VERSION}-fpm

Restart Nginx: systemctl restart nginx

Restart Workers: supervisorctl restart all

Clear Cache: cd \${INSTALL_PATH} && sudo -u www-data php artisan optimize:clear

Re-cache: cd \${INSTALL_PATH} && sudo -u www-data php artisan config:cache && sudo -u www-data php artisan route:cache && sudo -u www-data php artisan view:cache

Queue Status: supervisorctl status

=====

KEEP THIS FILE SECURE AND DELETE AFTER SAVING CREDENTIALS!

=====

EOF

```
chmod 600 $CREDS_FILE
log_info "Credentials saved to $CREDS_FILE"
```

```
}

print_completion() {
echo ""
echo -e "${GREEN}          ${NC}"
echo -e "${GREEN} UNIT3D Installation Complete! ${NC}"
echo -e "${GREEN}          ${NC}"
echo ""
echo -e "Your tracker is now available at: ${BLUE}https://${DOMAIN}${NC}"
echo ""
echo -e "Login with:"
echo -e " Username: ${YELLOW}${OWNER_USERNAME}${NC}"
echo -e " Password: ${YELLOW}${OWNER_PASSWORD}${NC}"
echo ""
}
```

```
echo -e "Credentials saved to: ${YELLOW}/root/unit3d-credentials.txt${NC}"
echo ""
echo -e "${RED}IMPORTANT: Save your credentials and delete the file!${NC}"
echo ""
}
```



MAIN



```
main() {
  print_banner
  check_root
  check_os
  interactive_setup
```

```
log_info "Starting UNIT3D installation..."

install_dependencies
configure_mysql
configure_redis
configure_meilisearch
configure_php
configure_nginx
install_unit3d
configure_laravel_echo_server
configure_supervisor
configure_cron

# Ask about SSL
echo ""
read -p "Set up SSL with Let's Encrypt? [Y/n]: " ssl_confirm
if [[ "$ssl_confirm" != "n" && "$ssl_confirm" != "N" ]]; then
  setup_ssl
fi

save_credentials
print_completion
```

```
}
```

Run main function

